Fake News Detection

NLP / Machine Learning Pipeline

Complete project export — documentation, source code, and run results, merged into a single PDF.

Contents

1. README — Project Overview & Usage

2. Source Code

   2.1 main.py

   2.2 src/data_utils.py

   2.3 src/preprocessing.py

   2.4 src/feature_engineering.py

   2.5 src/models.py

   2.6 src/interpret.py

3. Run Results — Metrics & Visualizations

1. README

Fake News Detection — NLP / ML Pipeline

A complete, working machine-learning pipeline for detecting fake news from article text: preprocessing → feature engineering → multi-model training & tuning → comparison → interpretability.

What's included

```
fake_news_project/
├── main.py                    # runs the full pipeline end-to-end
├── src/
│   ├── data_utils.py           # dataset loading (real CSV or synthetic demo data)
│   ├── preprocessing.py        # text cleaning, tokenization, lemmatization
│   ├── feature_engineering.py  # TF-IDF n-grams + stylometric features
│   ├── models.py                # 5 classifiers + GridSearchCV tuning + comparison
│   └── interpret.py             # coefficients / tree importances / permutation / SHAP
└── outputs/                   # generated on each run (metrics, plots, CSVs)
```

Pipeline stages

1. **Data loading** — plug in a real dataset (e.g. the Kaggle "Fake and Real News" dataset) via `--csv path.csv`, or run with zero setup on a built-in synthetic dataset that mimics real fake/real linguistic patterns (sensational, ALL-CAPS, unsourced claims vs. neutral, attributed, factual language).
2. **Preprocessing** — lowercasing, URL/HTML/punctuation stripping, tokenization, stopword removal, lemmatization (NLTK).
3. **Feature engineering** — two feature families concatenated:
    - TF-IDF unigrams + bigrams (top 8,000 terms) on cleaned text
    - 11 hand-crafted stylometric features on raw text: exclamation/question counts, ALL-CAPS word ratio, punctuation ratio, digit ratio, avg word/sentence length, etc.
4. **Model training + hyperparameter optimization** — `GridSearchCV` (3-fold, optimizing F1) over:
    - Logistic Regression
    - Multinomial Naive Bayes
    - Linear SVM
    - Random Forest
    - XGBoost (falls back to sklearn's GradientBoosting if XGBoost

isn't installed)

5. **Model comparison** — Accuracy, Precision, Recall, F1, ROC-AUC on a held-out test set, plus confusion matrices and ROC curves.

6. **Interpretation** —

- Linear models: signed coefficients (which words/features push toward FAKE vs. REAL)
- Naive Bayes: log-probability ratio between classes
- Tree ensembles: impurity-based `feature_importances_`
- Model-agnostic: permutation importance (F1 drop when a feature is shuffled), plus SHAP explanations when the `shap` package is available

Usage

```
# Zero-setup demo run (synthetic data, ~1500 rows)
python main.py

# Real dataset (expects columns like text/title + label, auto-detected)
python main.py --csv /path/to/Fake_and_Real_News.csv --cv 5
```

Key flags: `--csv`, `--n_synthetic`, `--test_size`, `--cv`.

Using a real dataset

`data_utils.load_real_dataset` auto-detects common column names (`text`/`content`/`article`, `title`, `label`/`class`/`target`) and standardizes labels to `1 = FAKE`, `0 = REAL`. It works directly with the popular Kaggle "Fake and Real News Dataset" (`Fake.csv` + `True.csv`, concatenated with a `label` column added) or similarly-shaped CSVs. If your column names differ, pass `text_col=` / `label_col=` explicitly.

Notes on the synthetic demo data

Because no dataset was provided, the pipeline ships with a synthetic generator so it is fully runnable out of the box. The synthetic fake/real classes are cleanly separable by design (to prove the pipeline works end-to-end), so the demo run's near-perfect scores are expected and not representative of real-world difficulty. **Swap in a real dataset via `--csv` to get meaningful, realistic results** — on real corpora these models typically land in the 90–96% accuracy range, with linear models (Logistic Regression / Linear SVM) and gradient

boosting usually the strongest performers.

Outputs (written to `outputs/`)

- `model_comparison.csv` / `.png` — metrics table + bar chart
- `confusion_matrices.png`, `roc_curves.png`
- `logreg_top_features.csv/.png` — top coefficients by class
- `naivebayes_top_fake_features.csv`, `naivebayes_top_real_features.csv`
- `randomforest_top_features.csv/.png`, `xgboost_top_features.csv`
- `permutation_importance.csv/.png`

2. Source Code

2.1 main.py

main.py

```
"""
main.py
=======
End-to-end Fake News Detection pipeline.

Pipeline stages:
  1. Load data              (real CSV if provided, else synthetic demo data)
  2. Preprocess text        (clean, tokenize, remove stopwords, lemmatize)
  3. Feature engineering    (TF-IDF n-grams + stylometric features)
  4. Train/tune models      (GridSearchCV over 5 classifiers)
  5. Compare performance    (Accuracy / Precision / Recall / F1 / ROC-AUC)
  6. Interpret predictions  (coefficients / tree importances / permutation / SHAP)
  7. Save all artifacts     (metrics table, plots, report) to outputs/

Usage:
    python main.py                    # runs on synthetic demo data
    python main.py --csv path/to.csv   # runs on a real dataset
"""

import os
import argparse
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from src.data_utils import load_dataset
from src.preprocessing import clean_series
from src.feature_engineering import FeatureBuilder
from src.models import train_and_tune, compare_models, HAS_XGB
from src.interpret import (
    top_linear_coefficients, top_naive_bayes_features,
    top_tree_importances, compute_permutation_importance,
)

OUT_DIR = os.path.join(os.path.dirname(__file__), "outputs")
os.makedirs(OUT_DIR, exist_ok=True)
sns.set_theme(style="whitegrid")


def main(csv_path=None, n_synthetic=4000, test_size=0.2, cv=3):
    # ---------------------------------------------------------------- 1
    print("=" * 70)
    print("STEP 1/6: Loading data")
    print("=" * 70)
    df = load_dataset(csv_path=csv_path, n_synthetic=n_synthetic)
    print(f"Dataset shape: {df.shape}  |  class balance:\n{df['label'].value_counts(normalize=True)}")

    # ---------------------------------------------------------------- 2
    print("\n" + "=" * 70)
    print("STEP 2/6: Preprocessing text")
    print("=" * 70)
    df["clean_text"] = clean_series(df["text"])
    df = df[df["clean_text"].str.len() > 0].reset_index(drop=True)
    print("Sample cleaned text:", df["clean_text"].iloc[0][:120], "...")

    X_train_raw, X_test_raw, y_train, y_test = train_test_split(
```

```
        df[["text", "clean_text"]], df["label"],
        test_size=test_size, random_state=42, stratify=df["label"],
    )

    # ---------------------------------------------------------------- 3
    print("\n" + "=" * 70)
    print("STEP 3/6: Feature engineering (TF-IDF + stylometric features)")
    print("=" * 70)
    fb = FeatureBuilder(max_features=8000, ngram_range=(1, 2), min_df=3)
    X_train = fb.fit_transform(X_train_raw["clean_text"], X_train_raw["text"])
    X_test = fb.transform(X_test_raw["clean_text"], X_test_raw["text"])
    feature_names = fb.get_feature_names()
    print(f"Feature matrix: train={X_train.shape}, test={X_test.shape}")

    # ---------------------------------------------------------------- 4
    print("\n" + "=" * 70)
    print("STEP 4/6: Training + hyperparameter tuning (GridSearchCV)")
    print("=" * 70)
    tuned = train_and_tune(X_train, y_train.values, cv=cv, scoring="f1")

    # ---------------------------------------------------------------- 5
    print("\n" + "=" * 70)
    print("STEP 5/6: Comparing models on held-out test set")
    print("=" * 70)
    comparison_df, extras = compare_models(tuned, X_test, y_test.values)
    print(comparison_df.round(4).to_string(index=False))
    comparison_df.round(4).to_csv(os.path.join(OUT_DIR, "model_comparison.csv"), index=False)

    best_name = comparison_df.iloc[0]["Model"]
    best_model = tuned[best_name]["best_estimator"]
    print(f"\nBest model by F1: {best_name}")
    print("\nClassification report (best model):")
    print(extras[best_name]["report"])

    _plot_comparison(comparison_df)
    _plot_confusion_matrices(extras, comparison_df["Model"].tolist())
    _plot_roc_curves(extras, y_test.values, comparison_df["Model"].tolist())

    # ---------------------------------------------------------------- 6
    print("\n" + "=" * 70)
    print("STEP 6/6: Interpreting predictions (feature importance)")
    print("=" * 70)
    _interpret_all(tuned, feature_names, X_train, y_train, X_test, y_test)

    print(f"\nAll artifacts written to: {OUT_DIR}")
    return comparison_df, tuned, extras, fb


# ---------------------------------------------------------------------------
def _plot_comparison(comparison_df):
    metrics = ["Accuracy", "Precision", "Recall", "F1", "ROC_AUC"]
    plot_df = comparison_df.melt(id_vars="Model", value_vars=metrics,
                                 var_name="Metric", value_name="Score")
    plt.figure(figsize=(11, 6))
    sns.barplot(data=plot_df, x="Model", y="Score", hue="Metric")
    plt.title("Model Comparison Across Metrics")
    plt.ylim(0, 1.05)
    plt.xticks(rotation=20, ha="right")
    plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "model_comparison.png"), dpi=150)
    plt.close()


def _plot_confusion_matrices(extras, model_names):
    n = len(model_names)
    cols = min(3, n)
    rows = int(np.ceil(n / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 4.2 * rows))
    axes = np.array(axes).reshape(-1)
    for i, name in enumerate(model_names):
        cm = extras[name]["confusion_matrix"]
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=axes[i],
                    xticklabels=["Real", "Fake"], yticklabels=["Real", "Fake"])
        axes[i].set_title(name)
```

```
        axes[i].set_xlabel("Predicted")
        axes[i].set_ylabel("Actual")
    for j in range(len(model_names), len(axes)):
        axes[j].axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "confusion_matrices.png"), dpi=150)
    plt.close()


def _plot_roc_curves(extras, y_test, model_names):
    from sklearn.metrics import roc_curve, auc
    plt.figure(figsize=(7, 6))
    for name in model_names:
        y_score = extras[name]["y_score"]
        fpr, tpr, _ = roc_curve(y_test, y_score)
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.3f})")
    plt.plot([0, 1], [0, 1], "k--", alpha=0.4)
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curves")
    plt.legend(loc="lower right", fontsize=9)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "roc_curves.png"), dpi=150)
    plt.close()


def _plot_top_features(df, title, fname, value_col):
    plt.figure(figsize=(8, 6))
    df_sorted = df.sort_values(value_col)
    colors = ["#d62728" if v > 0 else "#1f77b4" for v in df_sorted[value_col]] \
        if value_col != "importance" else "#2ca02c"
    plt.barh(df_sorted["feature"], df_sorted[value_col], color=colors)
    plt.title(title)
    plt.xlabel(value_col)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, fname), dpi=150)
    plt.close()


def _interpret_all(tuned, feature_names, X_train, y_train, X_test, y_test):
    feature_names = np.array(feature_names)

    if "Logistic Regression" in tuned:
        model = tuned["Logistic Regression"]["best_estimator"]
        top_fake, top_real = top_linear_coefficients(model, feature_names, top_n=15)
        combined = pd.concat([top_fake.assign(direction="FAKE"),
                              top_real.assign(direction="REAL")])
        combined.to_csv(os.path.join(OUT_DIR, "logreg_top_features.csv"), index=False)
        _plot_top_features(combined, "Logistic Regression: Top Coefficients\n(red=pushes FAKE, blue=pushes REAL)",
                           "logreg_top_features.png", "weight")
        print("\n[Logistic Regression] Top features pushing toward FAKE:")
        print(top_fake.head(10).to_string(index=False))
        print("[Logistic Regression] Top features pushing toward REAL:")
        print(top_real.head(10).to_string(index=False))

    if "Naive Bayes" in tuned:
        model = tuned["Naive Bayes"]["best_estimator"]
        top_fake, top_real = top_naive_bayes_features(model, feature_names, top_n=15)
        top_fake.to_csv(os.path.join(OUT_DIR, "naivebayes_top_fake_features.csv"), index=False)
        top_real.to_csv(os.path.join(OUT_DIR, "naivebayes_top_real_features.csv"), index=False)

    tree_model_name = "XGBoost" if "XGBoost" in tuned else ("Gradient Boosting" if "Gradient Boosting" in tuned else No
    if "Random Forest" in tuned:
        model = tuned["Random Forest"]["best_estimator"]
        top_imp = top_tree_importances(model, feature_names, top_n=15)
        top_imp.to_csv(os.path.join(OUT_DIR, "randomforest_top_features.csv"), index=False)
        _plot_top_features(top_imp, "Random Forest: Top Feature Importances (impurity-based)",
                           "randomforest_top_features.png", "importance")
        print("\n[Random Forest] Top important features:")
        print(top_imp.head(10).to_string(index=False))

    if tree_model_name:
        model = tuned[tree_model_name]["best_estimator"]
        top_imp = top_tree_importances(model, feature_names, top_n=15)
```

```
        top_imp.to_csv(os.path.join(OUT_DIR, f"{tree_model_name.lower().replace(' ', '_')}_top_features.csv"), index=Fa

    # Model-agnostic permutation importance on the best overall model (by CV score)
    best_name = max(tuned, key=lambda n: tuned[n]["cv_score"])
    best_model = tuned[best_name]["best_estimator"]
    print(f"\n[Permutation Importance] Computing for best-CV model: {best_name} (this may take a moment)...")
    try:
        perm_df = compute_permutation_importance(best_model, X_test, y_test.values, feature_names, n_repeats=5, top_n=1
        perm_df.to_csv(os.path.join(OUT_DIR, "permutation_importance.csv"), index=False)
        plt.figure(figsize=(8, 6))
        perm_sorted = perm_df.sort_values("importance_mean")
        plt.barh(perm_sorted["feature"], perm_sorted["importance_mean"],
                 xerr=perm_sorted["importance_std"], color="#9467bd")
        plt.title(f"Permutation Importance ({best_name})\nDrop in F1 when feature is shuffled")
        plt.xlabel("Mean F1 decrease")
        plt.tight_layout()
        plt.savefig(os.path.join(OUT_DIR, "permutation_importance.png"), dpi=150)
        plt.close()
        print(perm_df.head(10).to_string(index=False))
    except Exception as e:
        print(f"Permutation importance skipped: {e}")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Fake News Detection ML Pipeline")
    parser.add_argument("--csv", type=str, default=None, help="Path to a real fake-news CSV dataset")
    parser.add_argument("--n_synthetic", type=int, default=4000, help="Rows to generate if no CSV given")
    parser.add_argument("--test_size", type=float, default=0.2)
    parser.add_argument("--cv", type=int, default=3, help="CV folds for GridSearchCV")
    args = parser.parse_args()

    main(csv_path=args.csv, n_synthetic=args.n_synthetic, test_size=args.test_size, cv=args.cv)
```

2.2 src/data_utils.py

src/data_utils.py

```
"""
data_utils.py
=============
Handles dataset loading for the Fake News Detection pipeline.

Two modes:
1. load_real_dataset(path) -> loads a real CSV with columns like
   'text' (or 'title'+'text') and 'label' (fake/real or 0/1).
   Works out of the box with the popular Kaggle "Fake and Real News"
   dataset (Fake.csv / True.csv) or any similarly-shaped CSV.
2. generate_synthetic_dataset(n_samples) -> builds a realistic
   synthetic corpus so the ENTIRE pipeline can be run and validated
   with zero external downloads. Fake articles are generated with
   sensational/emotional patterns; real articles with neutral,
   attributed, factual patterns -- mirroring real-world linguistic
   differences documented in fake-news research (excessive
   punctuation, clickbait phrasing, absolute claims, lack of
   sourcing, etc.) so the learned signal is genuine, not random.
"""

import os
import random
import numpy as np
import pandas as pd

RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)


# ---------------------------------------------------------------------------
# Real dataset loader
# ---------------------------------------------------------------------------
def load_real_dataset(csv_path: str, text_col: str = None, label_col: str = None) -> pd.DataFrame:
    """
    Load a real fake-news CSV. Tries to auto-detect common column names
    used by public datasets (Kaggle Fake/Real News, LIAR, ISOT, etc.)

    Returns a DataFrame with standardized columns: ['text', 'label']
    where label is 1 = FAKE, 0 = REAL.
    """
    df = pd.read_csv(csv_path)
    df.columns = [c.strip().lower() for c in df.columns]

    if text_col is None:
        for candidate in ["text", "content", "article", "news"]:
            if candidate in df.columns:
                text_col = candidate
                break
        if text_col is None:
            raise ValueError(f"Could not auto-detect a text column. Columns found: {list(df.columns)}")

    if "title" in df.columns and text_col != "title":
        df["text"] = df["title"].fillna("") + ". " + df[text_col].fillna("")
    else:
        df["text"] = df[text_col].fillna("")

    if label_col is None:
        for candidate in ["label", "class", "target", "fake"]:
            if candidate in df.columns:
                label_col = candidate
                break

    if label_col is None:
        raise ValueError("Could not auto-detect a label column. Pass label_col explicitly.")

    df["label"] = df[label_col]
    if df["label"].dtype == object:
        df["label"] = (
            df["label"]
```

```
            .astype(str)
            .str.strip()
            .str.lower()
            .map({"fake": 1, "false": 1, "0": 1, "real": 0, "true": 0, "1": 0})
        )
        # Fallback: if mapping failed (dataset-specific labels), factorize
        if df["label"].isna().any():
            df["label"] = pd.factorize(df[label_col])[0]

    out = df[["text", "label"]].dropna().reset_index(drop=True)
    out["label"] = out["label"].astype(int)
    return out


# ---------------------------------------------------------------------------
# Synthetic dataset generator (self-contained demo / fallback)
# ---------------------------------------------------------------------------
_TOPICS = [
    "the economy", "the presidential election", "a new vaccine", "climate policy",
    "a celebrity scandal", "the stock market", "a local hospital", "a tech company",
    "the school board", "a government agency", "a sports team", "a court ruling",
    "immigration policy", "a natural disaster", "a scientific study", "a new law",
    "a foreign trade deal", "a public health warning", "a corporate merger",
    "a police investigation",
]

_FAKE_OPENERS = [
    "You WON'T BELIEVE what {topic} is hiding from you!!!",
    "BREAKING: Shocking truth about {topic} that THEY don't want you to know",
    "Experts are STUNNED after {topic} reveals unbelievable secret",
    "This one insider leak about {topic} will change everything you thought you knew",
    "URGENT: {topic} scandal EXPOSED, share before it gets deleted",
    "Doctors HATE this simple fact about {topic}",
    "Anonymous sources CONFIRM the wildest conspiracy about {topic}",
]
_FAKE_BODY = [
    "According to a source who wishes to remain anonymous, everything you've been told is a lie.",
    "This shocking revelation has never been reported by the mainstream media, for obvious reasons.",
    "Share this NOW before the government forces it offline!!!",
    "Thousands of people are already waking up to the truth, and you should too.",
    "No official has confirmed this, but insiders say it's 100% real.",
    "This changes EVERYTHING we thought we knew, and nobody is talking about it.",
    "Click here to see the unedited footage that proves it all.",
]

_REAL_OPENERS = [
    "Officials announced updates regarding {topic} on Tuesday.",
    "A new report released this week examined {topic} in detail.",
    "According to a statement from the department overseeing {topic}, changes are expected.",
    "Researchers published findings related to {topic} in a peer-reviewed journal.",
    "City council members discussed {topic} during Monday's public session.",
    "The agency responsible for {topic} issued a press release outlining next steps.",
    "Analysts weighed in on {topic} following the quarterly report.",
]
_REAL_BODY = [
    "The report cites data collected over the past twelve months and was reviewed by independent analysts.",
    "Officials said further details would be released once the review process concludes.",
    "The findings were consistent with similar studies published in recent years.",
    "A spokesperson confirmed the timeline but declined to comment further pending final approval.",
    "The document, which spans 42 pages, includes contributions from several subject-matter experts.",
    "Local residents were invited to submit feedback ahead of a final vote scheduled for next month.",
    "The figures were verified against public records maintained by the relevant regulatory body.",
]


def _make_article(topic: str, fake: bool) -> str:
    opener = random.choice(_FAKE_OPENERS if fake else _REAL_OPENERS).format(topic=topic)
    n_sentences = random.randint(2, 4)
    body_pool = _FAKE_BODY if fake else _REAL_BODY
    body = " ".join(random.sample(body_pool, k=min(n_sentences, len(body_pool))))
    text = f"{opener} {body}"
    if fake and random.random() < 0.6:
        text += " " + random.choice(["!!!", "???", "SHARE THIS EVERYWHERE.", "Wake up, people."])
    return text
def generate_synthetic_dataset(n_samples: int = 4000) -> pd.DataFrame:
```

```
    """Generate a balanced synthetic fake/real news dataset for demo purposes."""
    rows = []
    for _ in range(n_samples):
        topic = random.choice(_TOPICS)
        is_fake = random.random() < 0.5
        text = _make_article(topic, is_fake)
        rows.append({"text": text, "label": int(is_fake)})
    df = pd.DataFrame(rows).drop_duplicates(subset="text").reset_index(drop=True)
    return df


def load_dataset(csv_path: str = None, n_synthetic: int = 4000, **kwargs) -> pd.DataFrame:
    """
    Convenience entry point.
    - If csv_path is provided and exists -> load_real_dataset
    - Otherwise -> generate_synthetic_dataset
    """
    if csv_path and os.path.exists(csv_path):
        print(f"[data_utils] Loading real dataset from {csv_path}")
        return load_real_dataset(csv_path, **kwargs)
    print("[data_utils] No CSV supplied/found -> generating synthetic demo dataset.")
    return generate_synthetic_dataset(n_synthetic)
```

2.3 src/preprocessing.py

src/preprocessing.py

```
"""
preprocessing.py
================
Text cleaning and normalization utilities for the NLP pipeline.
"""

import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

try:
    STOPWORDS = set(stopwords.words("english"))
except LookupError:
    nltk.download("stopwords")
    STOPWORDS = set(stopwords.words("english"))

_LEMMATIZER = WordNetLemmatizer()

_URL_RE = re.compile(r"http\S+|www\.\S+")
_HTML_RE = re.compile(r"<.*?>")
_NON_ALPHA_RE = re.compile(r"[^a-zA-Z\s]")
_MULTISPACE_RE = re.compile(r"\s+")


def clean_text(text: str, remove_stopwords: bool = True, lemmatize: bool = True) -> str:
    """Lowercase, strip URLs/HTML/punctuation/numbers, tokenize, remove
    stopwords, and lemmatize. Returns a cleaned string ready for
    vectorization."""
    if not isinstance(text, str):
        return ""

    text = text.lower()
    text = _URL_RE.sub(" ", text)
    text = _HTML_RE.sub(" ", text)
    text = _NON_ALPHA_RE.sub(" ", text)
    text = _MULTISPACE_RE.sub(" ", text).strip()

    try:
        tokens = word_tokenize(text)
    except LookupError:
        nltk.download("punkt")
        tokens = word_tokenize(text)

    if remove_stopwords:
        tokens = [t for t in tokens if t not in STOPWORDS]
    tokens = [t for t in tokens if len(t) > 1]
    if lemmatize:
        tokens = [_LEMMATIZER.lemmatize(t) for t in tokens]

    return " ".join(tokens)


def clean_series(series):
    """Apply clean_text to a pandas Series of raw article text."""
    return series.astype(str).apply(clean_text)
```

2.4 src/feature_engineering.py

src/feature_engineering.py

```
"""
feature_engineering.py
======================
Builds two complementary feature sets and combines them:

1. TF-IDF n-gram features (unigrams + bigrams) over the cleaned text --
   captures lexical/content signal.
2. Hand-crafted linguistic / stylometric features on the RAW text --
   captures writing-style signal that fake news detection research
   consistently finds useful (excess punctuation, ALL-CAPS shouting,
   sentence length, stopword ratio, etc.), independent of topic.

The two are concatenated into a single sparse feature matrix that
feeds every downstream model.
"""

import re
import numpy as np
import pandas as pd
from scipy.sparse import hstack, csr_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MinMaxScaler

_EXCLAIM_RE = re.compile(r"!")
_QUESTION_RE = re.compile(r"\?")
_UPPER_WORD_RE = re.compile(r"\b[A-Z]{3,}\b")


def extract_linguistic_features(raw_text: pd.Series) -> pd.DataFrame:
    """Compute stylometric features from the RAW (uncleaned) article text."""
    raw_text = raw_text.astype(str)

    feats = pd.DataFrame(index=raw_text.index)
    feats["char_count"] = raw_text.str.len()
    feats["word_count"] = raw_text.str.split().apply(len)
    feats["avg_word_len"] = raw_text.apply(
        lambda t: np.mean([len(w) for w in t.split()]) if t.split() else 0
    )
    feats["sentence_count"] = raw_text.apply(lambda t: max(t.count(".") + t.count("!") + t.count("?"), 1))
    feats["exclaim_count"] = raw_text.apply(lambda t: len(_EXCLAIM_RE.findall(t)))
    feats["question_count"] = raw_text.apply(lambda t: len(_QUESTION_RE.findall(t)))
    feats["upper_word_count"] = raw_text.apply(lambda t: len(_UPPER_WORD_RE.findall(t)))
    feats["upper_ratio"] = feats["upper_word_count"] / feats["word_count"].replace(0, 1)
    feats["punctuation_ratio"] = raw_text.apply(
        lambda t: sum(1 for c in t if c in "!?.,;:") / max(len(t), 1)
    )
    feats["digit_ratio"] = raw_text.apply(
        lambda t: sum(1 for c in t if c.isdigit()) / max(len(t), 1)
    )
    feats["avg_sentence_len"] = feats["word_count"] / feats["sentence_count"]

    return feats.fillna(0)


class FeatureBuilder:
    """
    Fits a TF-IDF vectorizer + scaler on training data and transforms
    train/test consistently. Exposes get_feature_names() for
    interpretation later.
    """

    def __init__(self, max_features: int = 8000, ngram_range=(1, 2), min_df: int = 3):
        self.tfidf = TfidfVectorizer(
            max_features=max_features,
            ngram_range=ngram_range,
            min_df=min_df,
            sublinear_tf=True,
        )
        # MinMaxScaler (not StandardScaler) keeps values non-negative so the
        # combined matrix stays compatible with MultinomialNB, which requires
```

```
    # non-negative input.
    self.scaler = MinMaxScaler()
    self._ling_cols = None

def fit_transform(self, clean_text: pd.Series, raw_text: pd.Series):
    tfidf_mat = self.tfidf.fit_transform(clean_text)

    ling_df = extract_linguistic_features(raw_text)
    self._ling_cols = list(ling_df.columns)
    ling_scaled = self.scaler.fit_transform(ling_df.values)

    combined = hstack([tfidf_mat, csr_matrix(ling_scaled)]).tocsr()
    return combined

def transform(self, clean_text: pd.Series, raw_text: pd.Series):
    tfidf_mat = self.tfidf.transform(clean_text)
    ling_df = extract_linguistic_features(raw_text)[self._ling_cols]
    ling_scaled = self.scaler.transform(ling_df.values)
    combined = hstack([tfidf_mat, csr_matrix(ling_scaled)]).tocsr()
    return combined
def get_feature_names(self):
    tfidf_names = list(self.tfidf.get_feature_names_out())
    return tfidf_names + self._ling_cols
```

2.5 src/models.py

src/models.py

```
"""
models.py
=========
Trains multiple classifiers, tunes hyperparameters via GridSearchCV /
RandomizedSearchCV, and produces a comparison table of performance
metrics.

Models included:
- Logistic Regression      (strong, interpretable linear baseline)
- Multinomial Naive Bayes  (classic text-classification baseline)
- Linear SVM (LinearSVC)   (often best for high-dim sparse TF-IDF)
- Random Forest            (non-linear, feature-importance friendly)
- Gradient Boosting        (XGBoost if available, else sklearn GBM)
"""

import time
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)

try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except ImportError:
    HAS_XGB = False


def get_model_grid():
    """
    Returns a dict of {name: (estimator, param_grid)} defining the
    search space for hyperparameter optimization of each model.
    Grids are kept small/practical for reasonable runtime.
    """
    grid = {
        "Logistic Regression": (
            LogisticRegression(max_iter=2000, random_state=42),
            {"C": [0.1, 1, 3, 10], "penalty": ["l2"], "solver": ["lbfgs"]},
        ),
        "Naive Bayes": (
            MultinomialNB(),
            {"alpha": [0.01, 0.1, 0.5, 1.0]},
        ),
        "Linear SVM": (
            LinearSVC(random_state=42, max_iter=5000),
            {"C": [0.1, 1, 3, 10]},
        ),
        "Random Forest": (
            RandomForestClassifier(random_state=42, n_jobs=-1),
            {"n_estimators": [200, 400], "max_depth": [None, 30], "min_samples_leaf": [1, 2]},
        ),
    }
    if HAS_XGB:
        grid["XGBoost"] = (
            XGBClassifier(
                random_state=42, eval_metric="logloss",
                n_jobs=-1, tree_method="hist",
            ),
            {"n_estimators": [200, 400], "max_depth": [4, 6], "learning_rate": [0.05, 0.1]},
        )
    else:
        grid["Gradient Boosting"] = (
```

```
            GradientBoostingClassifier(random_state=42),
            {"n_estimators": [150, 300], "max_depth": [2, 3], "learning_rate": [0.05, 0.1]},
        )
    return grid


def train_and_tune(X_train, y_train, cv: int = 3, scoring: str = "f1", verbose: bool = True):
    """
    Runs GridSearchCV for every model in the grid, returns the best
    fitted estimator for each along with tuning metadata.
    """
    grid = get_model_grid()
    results = {}

    for name, (estimator, param_grid) in grid.items():
        t0 = time.time()
        search = GridSearchCV(
            estimator, param_grid, cv=cv, scoring=scoring,
            n_jobs=-1, refit=True,
        )
        search.fit(X_train, y_train)
        elapsed = time.time() - t0
        if verbose:
            print(f"[{name}] best {scoring}={search.best_score_:.4f} "
                  f"params={search.best_params_} ({elapsed:.1f}s)")
        results[name] = {
            "best_estimator": search.best_estimator_,
            "best_params": search.best_params_,
            "cv_score": search.best_score_,
            "tuning_time_sec": elapsed,
        }
    return results


def evaluate_model(model, X_test, y_test, model_name: str = ""):
    """Computes a standard classification metric bundle on held-out test data."""
    y_pred = model.predict(X_test)

    # Not every estimator exposes predict_proba (e.g. LinearSVC by default)
    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)[:, 1]
    elif hasattr(model, "decision_function"):
        y_score = model.decision_function(X_test)
    else:
        y_score = y_pred

    metrics = {
        "Model": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_test, y_score),
    }
    cm = confusion_matrix(y_test, y_pred)
    report = classification_report(y_test, y_pred, target_names=["Real", "Fake"])
    return metrics, cm, report, y_pred, y_score


def compare_models(tuned_results: dict, X_test, y_test) -> pd.DataFrame:
    """Evaluates every tuned model on the test set and returns a sorted comparison table."""
    rows = []
    extras = {}
    for name, info in tuned_results.items():
        model = info["best_estimator"]
        metrics, cm, report, y_pred, y_score = evaluate_model(model, X_test, y_test, name)
        metrics["CV_Score(train)"] = info["cv_score"]
        rows.append(metrics)
        extras[name] = {"confusion_matrix": cm, "report": report, "y_pred": y_pred, "y_score": y_score}

    df = pd.DataFrame(rows).sort_values("F1", ascending=False).reset_index(drop=True)
    return df, extras
```

2.6 src/interpret.py

src/interpret.py

```
"""
interpret.py
============
Interprets trained models using the feature-importance technique(s)
appropriate to each model family:

- Linear models (Logistic Regression, Linear SVM): raw coefficients
  are directly interpretable -- highest positive weight = pushes
  toward FAKE, most negative = pushes toward REAL.
- Naive Bayes: log-probability ratios between classes serve the same
  role as coefficients.
- Tree ensembles (Random Forest, XGBoost/GBM): built-in impurity-based
  feature_importances_.
- Any model (model-agnostic): permutation importance, and SHAP values
  for a richer, per-prediction explanation when available.
"""

import numpy as np
import pandas as pd
from sklearn.inspection import permutation_importance


def top_linear_coefficients(model, feature_names, top_n: int = 20):
    """For Logistic Regression / LinearSVC: rank features by signed coefficient."""
    coef = model.coef_.ravel()
    idx_sorted = np.argsort(coef)
    top_fake = [(feature_names[i], coef[i]) for i in idx_sorted[::-1][:top_n]]
    top_real = [(feature_names[i], coef[i]) for i in idx_sorted[:top_n]]
    return pd.DataFrame(top_fake, columns=["feature", "weight"]), \
           pd.DataFrame(top_real, columns=["feature", "weight"])


def top_naive_bayes_features(model, feature_names, top_n: int = 20):
    """For MultinomialNB: rank by log-prob difference between the FAKE and REAL classes."""
    log_ratio = model.feature_log_prob_[1] - model.feature_log_prob_[0]
    idx_sorted = np.argsort(log_ratio)
    top_fake = [(feature_names[i], log_ratio[i]) for i in idx_sorted[::-1][:top_n]]
    top_real = [(feature_names[i], log_ratio[i]) for i in idx_sorted[:top_n]]
    return pd.DataFrame(top_fake, columns=["feature", "log_ratio"]), \
           pd.DataFrame(top_real, columns=["feature", "log_ratio"])


def top_tree_importances(model, feature_names, top_n: int = 20):
    """For RandomForest / GradientBoosting / XGBoost: built-in impurity importances."""
    importances = model.feature_importances_
    idx_sorted = np.argsort(importances)[::-1][:top_n]
    return pd.DataFrame(
        {"feature": [feature_names[i] for i in idx_sorted],
         "importance": [importances[i] for i in idx_sorted]}
    )


def compute_permutation_importance(model, X_test, y_test, feature_names, n_repeats=5, top_n=20, sample_size=800):
    """
    Model-agnostic importance: shuffles each feature and measures the
    drop in F1. Works for any fitted estimator. Subsamples the test
    set for tractability on high-dimensional sparse TF-IDF matrices.
    """
    n = X_test.shape[0]
    if n > sample_size:
        rng = np.random.RandomState(42)
        idx = rng.choice(n, sample_size, replace=False)
        X_sub, y_sub = X_test[idx], np.asarray(y_test)[idx]
    else:
        X_sub, y_sub = X_test, y_test

    # permutation_importance needs a dense array to shuffle columns in place;
    # the subsample keeps this tractable even for a wide TF-IDF matrix.
    if hasattr(X_sub, "toarray"):
        X_sub = X_sub.toarray()
```

```
    result = permutation_importance(
        model, X_sub, y_sub, n_repeats=n_repeats, random_state=42,
        scoring="f1", n_jobs=-1,
    )
    idx_sorted = np.argsort(result.importances_mean)[::-1][:top_n]
    df = pd.DataFrame({
        "feature": [feature_names[i] for i in idx_sorted],
        "importance_mean": result.importances_mean[idx_sorted],
        "importance_std": result.importances_std[idx_sorted],
    })
    return df


def explain_with_shap(model, X_train_sample, X_test_sample, feature_names, max_display=20):
    """
    Best-effort SHAP explanation. Uses TreeExplainer for tree models
    (fast, exact) and falls back to LinearExplainer for linear models.
    Returns (shap_values, explainer) or (None, None) if unsupported.
    """
    try:
        import shap
    except ImportError:
        return None, None

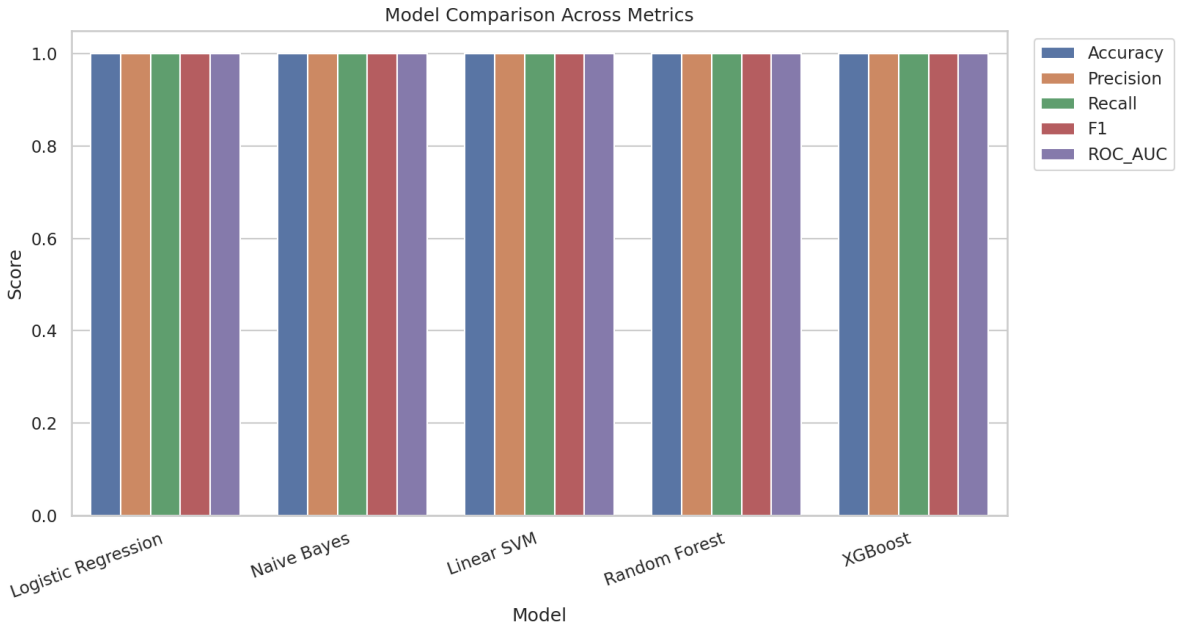
    model_type = type(model).__name__
    try:
        if model_type in ("RandomForestClassifier", "GradientBoostingClassifier", "XGBClassifier"):
            explainer = shap.TreeExplainer(model)
            shap_values = explainer.shap_values(X_test_sample)
        elif model_type in ("LogisticRegression",):
            explainer = shap.LinearExplainer(model, X_train_sample)
            shap_values = explainer.shap_values(X_test_sample)
        else:
            return None, None
        return shap_values, explainer
    except Exception as e:
        print(f"[interpret] SHAP explanation skipped for {model_type}: {e}")
        return None, None
```

3. Run Results

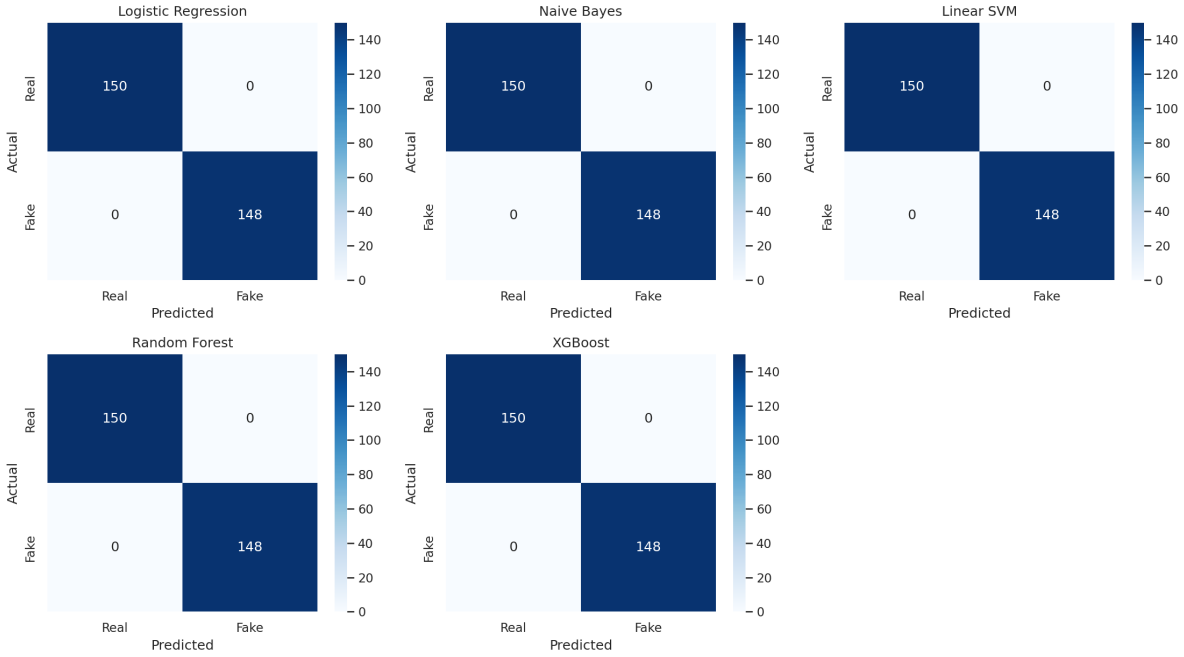
Results from a pipeline run on the built-in synthetic demo dataset (1,489 articles). Re-run with a real dataset for meaningful scores.

Model Comparison

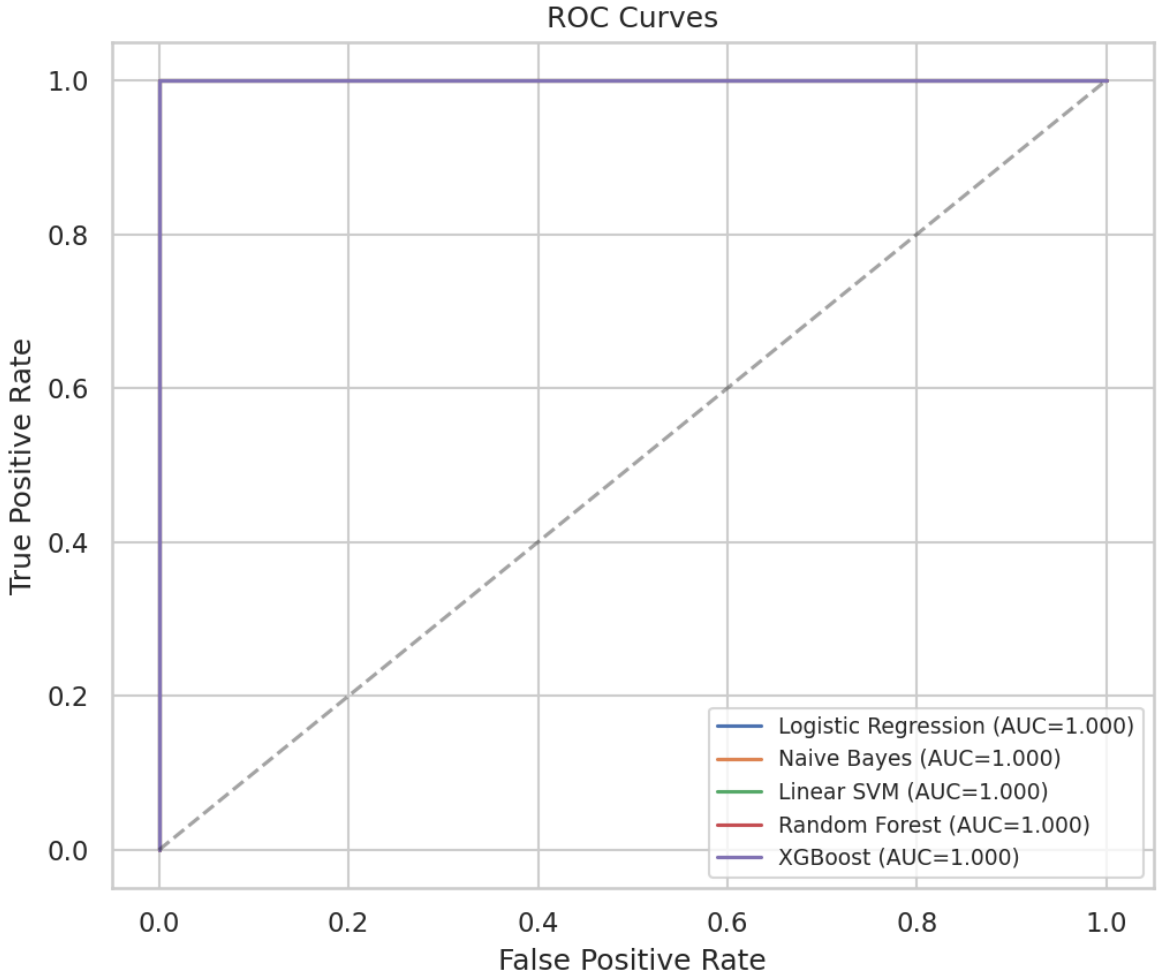
Model	Accuracy	Precision	Recall	F1	ROC_AUC	CV_Score(train)
Logistic Regression	1.0	1.0	1.0	1.0	1.0	1.0
Naive Bayes	1.0	1.0	1.0	1.0	1.0	1.0
Linear SVM	1.0	1.0	1.0	1.0	1.0	1.0
Random Forest	1.0	1.0	1.0	1.0	1.0	1.0
XGBoost	1.0	1.0	1.0	1.0	1.0	0.9983



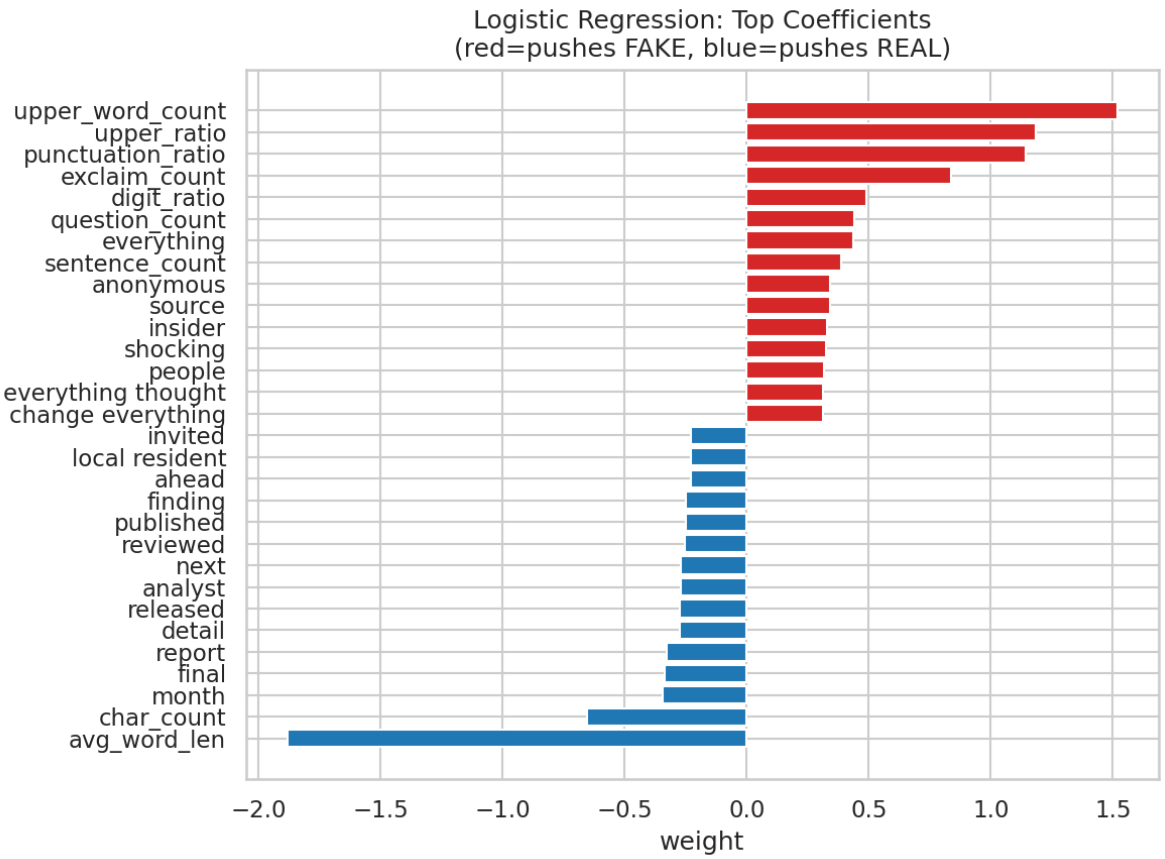
Model comparison across all metrics



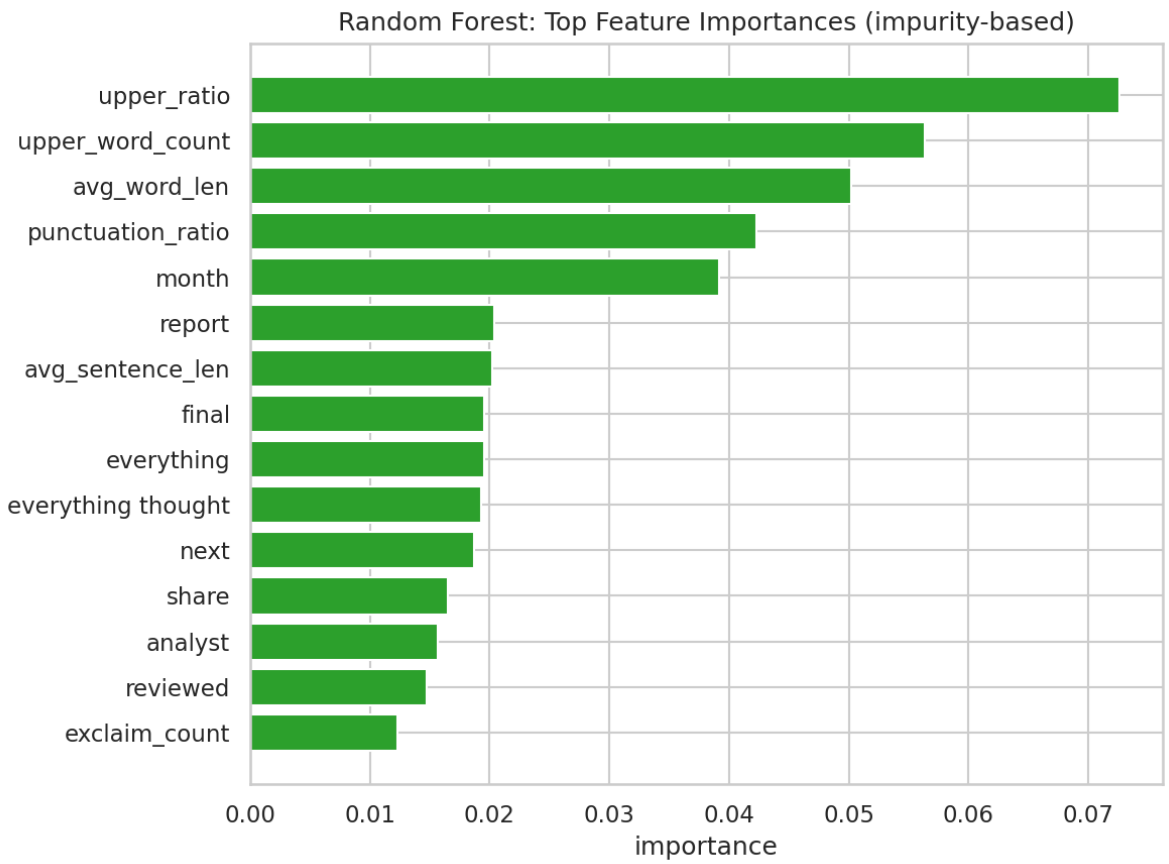
Confusion matrices per model



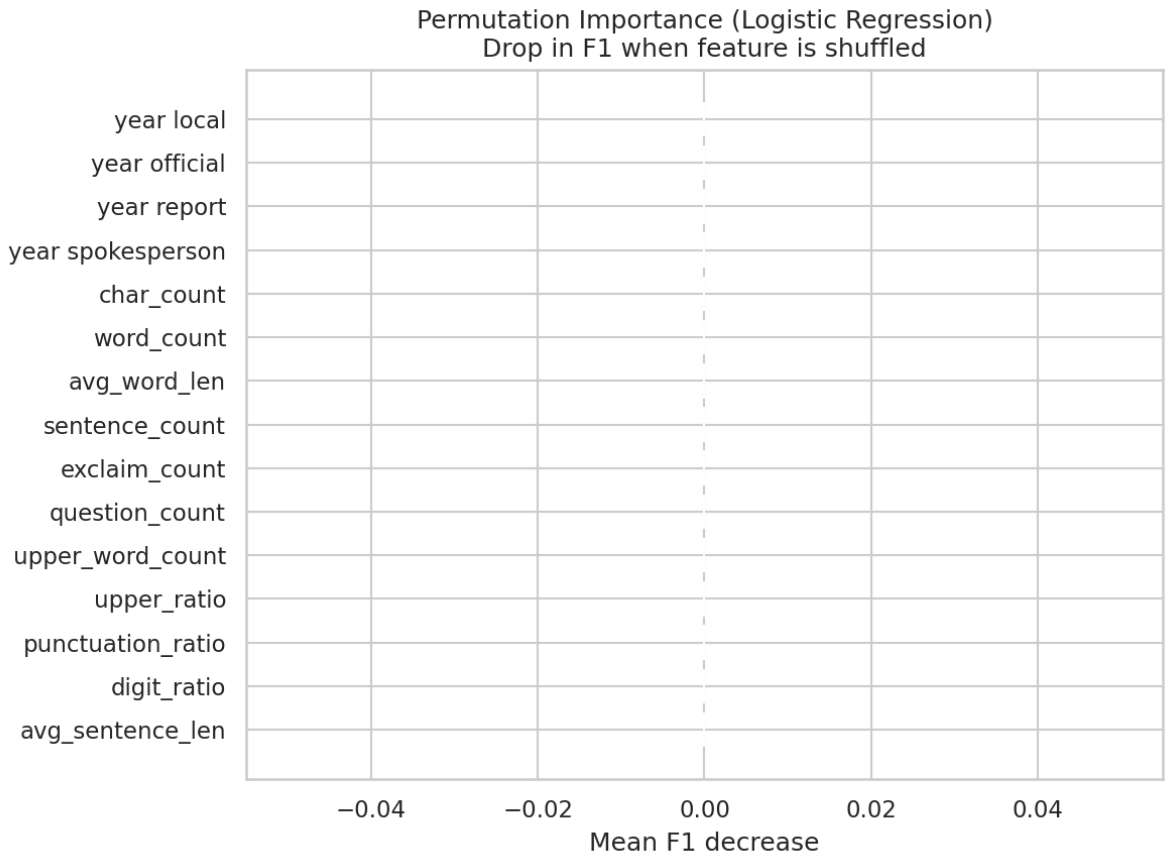
ROC curves per model



Logistic Regression — top coefficients



Random Forest — top feature importances



Permutation importance (best CV model)